

Shell Scripting – **case** Statement in Bash

Concept: **case** Statement in Bash

The **case** statement in Bash is used for **pattern matching** and **conditional execution**. It's similar to **switch** statements in other programming languages like C, Java, or JavaScript.

Syntax Overview

bash

```
case <variable> in
    pattern1)
        command1
        ;;
    pattern2)
        command2
        ;;
    ...
esac
```

- **case** begins the block.
- **<variable>** is the value being matched.
- Each **pattern)** is checked against the variable.
- If matched, the corresponding command(s) are executed.
- **;;** ends each case block.
- **esac** ends the entire **case** structure (it's **case** spelled backward).

Your Example Explained

bash

```
Name="Girish"
```

```
case "$Name" in
    "Rajib") echo "Profession : Software Engineer" ;;
    "Vikas") echo "Profession : Web Developer" ;;
    "Girish") echo "Profession : Technical Content Writer" ;;
esac
```

- The variable `Name` is set to `"Girish"`.
- The `case` statement checks the value of `Name`.
- It matches `"Girish"` and executes: `echo "Profession : Technical Content Writer"`.

✂ Use Cases of `case` Statement

- Menu-driven programs
- Command-line utilities
- Handling user input
- Automating decision-based tasks

Would you like a few practice problems or a mini quiz to reinforce this concept?

Decision-Making Statements in Shell Scripting

✂ Overview

Shell scripts use decision-making statements to control the flow of execution based on conditions. The two primary types are:

1 ☐ `if-else` Statement

Used to evaluate conditions and execute code blocks accordingly.

2 ☐ `case-esac` Statement

Used for pattern matching, especially when multiple conditions are involved.

🔍 1. `if-else` Statement Variants

✅ Basic `if-fi`

bash

```
Name="Girish"
if [ "$Name" = "Girish" ]; then
    echo "His name is Girish. It is true."
fi
```

- Executes only if the condition is true.
- Ends with `fi` (reverse of `if`).

✓ **if-else-fi**

bash

```
Age=17
if [ "$Age" -ge 18 ]; then
    echo "You can vote"
else
    echo "You cannot vote"
fi
```

- Provides an alternative path if the condition is false.

✓ **if-elif-else-fi**

bash

```
Age=17
if [ "$Age" -ge 18 ]; then
    echo "You can vote"
elif [ "$Age" -eq 17 ]; then
    echo "You can vote after one year"
else
    echo "You cannot vote"
fi
```

- Allows multiple conditions to be checked sequentially.

✓ **Nested if-else**

bash

```
echo "Enter subject"
read subject

if [ "$subject" == "Linux" ]; then
    echo "Enter Marks"
    read marks
    if [ "$marks" -ge 30 ]; then
        echo "You passed"
    else
        echo "You failed"
    fi
else
    echo "Wrong Subject"
fi
```

- One `if` block inside another.
- Useful for complex logic.

2. `case-esac` Statement

Used when you have multiple fixed options to match against a variable.

bash

```
Name="Girish"
case "$Name" in
  "Rajib") echo "Profession : Software Engineer" ;;
  "Vikas") echo "Profession : Web Developer" ;;
  "Girish") echo "Profession : Technical Content Writer" ;;
esac
```

- Each pattern ends with `;;`.
- Ends with `esac` (reverse of `case`).

When to Use What?

Use Case	Best Choice
Simple true/false check	<code>if-fi</code>
Multiple numeric or logical conditions	<code>if-elif-else-fi</code>
Matching fixed string values	<code>case-esac</code>
Complex nested logic	Nested <code>if-else</code>

Bash Scripting – Special Parameters and Positional Variables

Concept Overview

Bash provides **special variables** that give you information about the script's execution, arguments passed, and shell environment. These are especially useful when writing scripts that need to process user input or interact with the system.

Positional Parameters and Special Variables

Variable	Meaning
<code>\$0</code>	Name of the script file
<code>\$1, \$2, ...</code>	First, second, etc., arguments passed to the script
<code>\$#</code>	Total number of arguments passed
<code>\$*</code>	All arguments as a single string
<code>\$@</code>	All arguments as separate strings (quoted individually)
<code>_</code>	Last argument to the previous command
<code>?</code>	Exit status of the last executed command
<code>\$\$</code>	Process ID (PID) of the current shell

`$-` Current shell flags (e.g., `-x`, `-v`, `-e`)

Example Breakdown

```
bash
```

```
#!/bin/bash
```

```
echo "Number of argument passed: $#"  
echo "Script name is $0"  
echo "The 2nd argument passed is: $2"  
echo "Arguments passed to script are: $*"  
echo "Exit status of last command that executed: $?"  
echo "Last argument provided to previous command: $_"  
echo "PID of current shell is: $$"  
echo "Flags are set in the shell: $-"
```

What Happens When You Run It?

If you run the script like this:

```
bash
```

```
./myscript.sh Hello World
```

You'll get output like:

Code

```
Number of argument passed: 2  
Script name is ./myscript.sh  
The 2nd argument passed is: World  
Arguments passed to script are: Hello World  
Exit status of last command that executed: 0  
Last argument provided to previous command: World  
PID of current shell is: 12345  
Flags are set in the shell: himBH
```

Key Learning Points

- Use `$#` to validate input count.
- Use `$?` to check if a command succeeded.

- Use `$_` for quick access to the last argument.
- Use `$-` to debug or inspect shell behavior.

Would you like a mini quiz or practice exercise to reinforce this topic? I can also show how to use these in real-world scripts like automation or logging.

Shell Scripting – Variables and Their Management

Concepts Covered

1 Defining Variables

In Bash, variables are defined without spaces around the `=` sign:

```
bash

num="1"
name="Devil"
```

- Variable names are case-sensitive.
- No `$` when defining, but `$` is used when accessing.

2 Accessing Variables

Use `$` to retrieve the value:

```
bash

VAR_1="Devil"
VAR_2="OWL"
echo "$VAR_1$VAR_2" # Output: DevilOWL
```

3 Unsetting Variables

Use `unset` to delete a variable:

```
bash

var1="Devil"
var2=23
echo $var1 $var2
```

```
unset var1
echo $var1 $var2 # var1 is now empty
```

⚠ `unset` does **not** work on read-only variables.

4 Read-Only Variables

Use `readonly` to protect a variable from modification:

```
bash

var1="Devil"
readonly var1
echo $var1

var1="NewValue" # This will cause an error
```

5 Putting It All Together

Here's a combined example:

```
bash

Var_name="Devil"
Var_age=23

echo "Name is $Var_name, and age is $Var_age."

# Read-only variable
var_blood_group="O-"
readonly var_blood_group
echo "Blood group is $var_blood_group and read only."
var_blood_group="B+" # Error: cannot modify read-only variable

# Unsetting variable
unset Var_age
echo "After unsetting Var_age..."
echo "Name is $Var_name, blood group is $var_blood_group and age is $Var_age..."
```

Key Takeaways

Feature	Command	Behavior
---------	---------	----------

Define variable	<code>name="value"</code>	Stores data
Access variable	<code>\$name</code>	Retrieves value
Unset variable	<code>unset name</code>	Deletes variable
Read-only	<code>readonly name</code>	Locks variable

Would you like a mini quiz or a practical challenge to test your understanding of these concepts?

Bash Scripting – Conditional Expressions

Concept Overview

Conditional expressions allow you to compare **strings** and **numbers** in Bash. These are used inside `if` statements to control the flow of your script.

String Comparison Operators

Operator	Meaning	Example	Output
<code>==</code>	Equal	<code>'Girish' == 'Girish'</code>	<code>same</code>

<code>!=</code>	Not Equal	<code>'Girish' != 'Apple'</code>	not same
<code>-n</code>	Not null (non-empty string)	<code>-n "Girish"</code>	not null
<code>-z</code>	Null (empty string)	<code>-z "Girish"</code>	not null

✔ Use double quotes around strings to avoid errors with spaces or special characters.

Integer Comparison Operators

Operator	Meaning	Example	Output
<code>-eq</code>	Equal	<code>10 -eq 10</code>	Equal
<code>-ne</code>	Not Equal	<code>10 -ne 13</code>	Not Equal
<code>-gt</code>	Greater Than	<code>10 -gt 8</code>	Greater
<code>-ge</code>	Greater or Equal	<code>10 -ge 9</code>	Greater or equal
<code>-lt</code>	Less Than	<code>10 -lt 13</code>	Less than

<code>-le</code>	Less or Equal	<code>10 -le 12</code>	Less than or equal
------------------	---------------	------------------------	--------------------

✅ These are used only for **numeric comparisons**.

Best Practices

- Always quote variables: `"$var"` to avoid unexpected behavior.
- Use `[ ]` for simple conditions, and `[[ ]]` for extended pattern matching.
- For arithmetic, you can also use `(( ))` syntax: `if (( a > b ))`.